



INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS
CAMPUS MONTES CLAROS -MG
CIÊNCIA DA COMPUTAÇÃO



SARAH EMANUELLE ALVES LINO
TAINARA RIBEIRO SANTOS

RELATÓRIO: Socket, RPC e RMI

Relatório apresentado ao Professor Danilo Teixeira Silva como parte das exigências de avaliação da disciplina de Sistemas Distribuídos do Curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros.

MONTES CLAROS - MG

2026

1. Introdução

O objetivo desta atividade foi implementar uma mesma aplicação em três tecnologias distintas de comunicação entre processos (Sockets, RPC (via gRPC) e RMI) para posteriormente podermos compará-las.

A aplicação que escolhemos foi uma calculadora distribuída no modelo cliente-servidor. O cliente envia uma operação com dois operandos (SOMA, SUB, MUL, DIV) e o servidor devolve o resultado. Esta escolha foi proposital: por ter uma lógica simples e idêntica nas três versões, permite isolar exatamente aquilo que muda de uma abordagem para a outra (a forma de comunicação em rede), sem que a complexidade da regra de negócio interfira na comparação.

Para tornar a análise o mais justa possível, as três versões mantêm a mesma experiência de uso. O utilizador digita comandos no formato "OPERACAO A B" (ex: "SOMA 3 5") e recebe "RESULTADO x" ou "ERRO ...". As três versões foram desenvolvidas em Java e os mesmos casos de erro (formato inválido, operando não numérico e divisão por zero) são tratados em todas elas.

2. Arquitetura cliente-servidor comum

Nas três versões, a arquitetura de alto nível é a mesma:

- Um servidor que fica em execução contínua a aguardar requisições.
- Um cliente interativo que lê os comandos introduzidos pelo teclado, envia-os ao servidor e exibe a resposta no ecrã.

Testes e Validação: Para garantir que os sistemas distribuídos funcionavam de fato em rede e não apenas localmente, compilamos e executamos os códigos em dois computadores. As três versões (Sockets, gRPC e RMI) funcionaram perfeitamente nas diferentes máquinas, provando que a comunicação ocorre com sucesso em ambientes reais.

O que difere entre as versões é o que trafega na rede e a quantidade de código de comunicação que precisamos de escrever manualmente.

3. Versão 1 — Sockets (TCP)

Implementação: O servidor abre um `ServerSocket` na porta 5000. Por cada cliente que se liga, o servidor trata a comunicação numa *thread* separada, o que permite suportar vários clientes simultaneamente. A troca de dados baseia-se num protocolo de texto definido pelo usuário. O cliente envia uma linha como "SOMA 3 5", e o servidor faz o *split* da *string*, converte os dados, calcula e responde "RESULTADO 8.0".

Características: É o nível de abstração mais baixo. O programador é responsável por abrir a conexão, formatar as mensagens, interpretar o texto e tratar os erros de rede. Não existe um contrato formal pré-definido, apenas um formato de texto combinado informalmente entre o cliente e o servidor.

4. Versão 2 — RPC (com gRPC)

Implementação: O núcleo desta versão é o ficheiro de contrato `calculadora.proto`, que descreve o serviço Calculadora e as mensagens de pedido e resposta (`OperacaoRequest` e `OperacaoResponse`). A partir deste ficheiro, a ferramenta de *build* (Maven) gera automaticamente o código de infraestrutura de rede. O cliente chama métodos através de um *stub* (ex: `stub.soma(req)`) como se fosse uma função local. A comunicação utiliza a porta 50051 sobre HTTP/2, com mensagens serializadas em Protobuf.

Características: Apresenta um nível de abstração médio-alto. Não é necessário manipular *bytes* ou interpretar texto manualmente. A maior diferença face ao RMI é que o contrato (`.proto`) é neutro em relação à linguagem de programação, permitindo criar um cliente em Python que comunique com este servidor em Java.

5. Versão 3 — RMI (Java Remote Method Invocation)

Implementação: Definimos uma interface remota (`Calculadora`) e o respetivo objeto de implementação (`CalculadoraImpl`). O servidor inicia um RMI Registry na porta 1099 e regista o objeto com um nome específico. O cliente pesquisa por esse nome (`lookup`), recebe uma referência e chama os métodos (`calc.soma(3, 5)`) de forma quase idêntica a uma chamada local.

Características: É a abordagem com o nível de abstração mais alto das três. O contrato é assegurado pela própria interface Java e verificado durante a compilação. Toda a serialização, passagem de parâmetros e até o relançamento de exceções no cliente (como a divisão por zero) são geridos automaticamente pela infraestrutura do RMI. A limitação é que obriga ambas as pontas a serem em Java.

6. Comparação entre as Abordagens

6.1. Diferenças de Abstração e Implementação

Existe uma clara escada de abstração: **Socket** → **gRPC** → **RMI**.

- **Socket:** A unidade de troca é texto simples. Lida-se diretamente com os fluxos de rede. (`saida.println("SOMA 3 5")`).
- **gRPC:** A unidade é uma chamada de procedimento remoto. O "encanamento" da rede é gerado com base no contrato. (`stub.soma(OperacaoRequest...)`).
- **RMI:** A unidade é uma chamada de método. A rede fica totalmente invisível no código cliente. (`calc.soma(3, 5)`).

6.2. Complexidade e Casos de Uso

- **Socket:** Simples conceptualmente e não requer dependências externas, mas é propício a falhas manuais de protocolo e erros que só surgem em tempo de execução. Ideal para protocolos de baixo nível ou otimizações extremas.
- **gRPC:** O código fica muito limpo, mas exige a configuração mais complexa (Maven, protoc, ficheiros .proto). É a solução padrão da indústria para microserviços modernos devido à interoperabilidade entre várias linguagens.
- **RMI:** Muito produtivo e sem dependências externas. Contudo, fica restrito a ecossistemas 100% Java. Excelente para comunicação entre serviços internos desenvolvidos exclusivamente nesta linguagem.

6.3. Quadro Comparativo

Aspecto	Socket	gRPC (RPC)	RMI
Contrato	Informal (texto)	Ficheiro .proto	Interface Java
Serialização	Manual (pelo programador)	Automática (Protobuf)	Automática (Java)
Tratamento de Erro	<i>Strings</i> personalizadas	Status padronizado	Exceções nativas
Interoperabilidade	Qualquer linguagem	Várias linguagens	Apenas Java
Dependências	Apenas o JDK	Maven, protoc	Apenas o JDK
Porta Utilizada	5000	50051	1099

7. Dificuldades Encontradas no Desenvolvimento

Durante o trabalho prático, lidamos com alguns desafios:

- **Modelo de dois processos:** Compreender que o servidor deve ficar bloqueado e em execução num terminal enquanto operamos o cliente num terminal à parte.
- **Configuração do Maven no gRPC:** Foi o maior obstáculo. O *build* inicial falhou porque o maven-compiler-plugin ignorava a versão do Java. Tivemos de fixar a versão 3.13.0 no ficheiro pom.xml. Além disso, a primeira execução foi morosa devido à transferência de dependências do Protobuf.
- **Mensagens RMI:** Notamos que certas impressões de sistema (como exceções) surgiam no cliente e não no servidor, o que exigiu habituação à forma como o RMI reflete os erros na rede.

8. Instruções de execução

- Socket

```
javac ServidorCalculadora.java ClienteCalculadora.java
java ServidorCalculadora      # Terminal 1
java ClienteCalculadora      # Terminal 2
```

- RMI

```
javac Calculadora.java CalculadoraImpl.java ServidorRMI.java ClienteRMI.java
java ServidorRMI      # Terminal 1
java ClienteRMI      # Terminal 2
```

- gRPC

```
mvn compile
mvn exec:java -Dexec.mainClass=calculadora.ServidorGRPC # Terminal 1
mvn exec:java -Dexec.mainClass=calculadora.ClienteGRPC  # Terminal 2
```

9. Conclusão

As três tecnologias cumpriram o propósito de comunicar um cliente e um servidor em rede, mas solucionam o problema com níveis de abstração distintos. Os Sockets entregam controle total à custa de muito trabalho manual e suscetibilidade a erros. O gRPC proporciona um contrato neutro, forte e muito eficiente, mas exige uma ferramenta de compilação pesada. Por fim, o RMI oferece a experiência de desenvolvimento mais próxima à de métodos locais, sendo altamente produtivo, mas aprisiona o projeto à linguagem Java. A escolha real num projeto dependeria sempre de fatores como a necessidade de interoperabilidade, desempenho e o ecossistema existente.